01. INTRODUCTION

Background of the Project

The Three.JS 3D Portfolio is an immersive, browser-based interactive portfolio web application built using React Three Fiber (R3F) — a React renderer for Three.js. The project was developed during a structured 120-hour internship programme at SRIDIX Technology LLP, Surat. Inspired by the award-winning portfolio design of David Heckhoff (david-hckh.com), the application presents a developer's personal brand through a fully animated 3D scene featuring a custom Ready Player Me avatar that dynamically responds to real-time user interaction.

Three.js is one of the most powerful and widely-used JavaScript libraries for creating hardware-accelerated 3D graphics directly in the browser via WebGL. By layering React Three Fiber on top of Three.js, developers gain the productivity benefits of React's declarative component model while retaining full access to Three.js's rendering pipeline.

Rationale of the Internship

The primary motivation for undertaking this project was to gain practical, production-grade experience in 3D web development — a highly differentiated and in-demand skill set in the modern front-end development landscape. As a Bachelor of Computer Application student, theoretical exposure to graphics and web technology needed to be reinforced through a real working project.

This internship project specifically targeted: (a) mastery of the MERN-adjacent R3F/Three.js ecosystem, (b) integration of third-party 3D asset pipelines (Ready Player Me and Mixamo), and (c) creation of a professional digital portfolio artefact that demonstrates advanced technical capability.

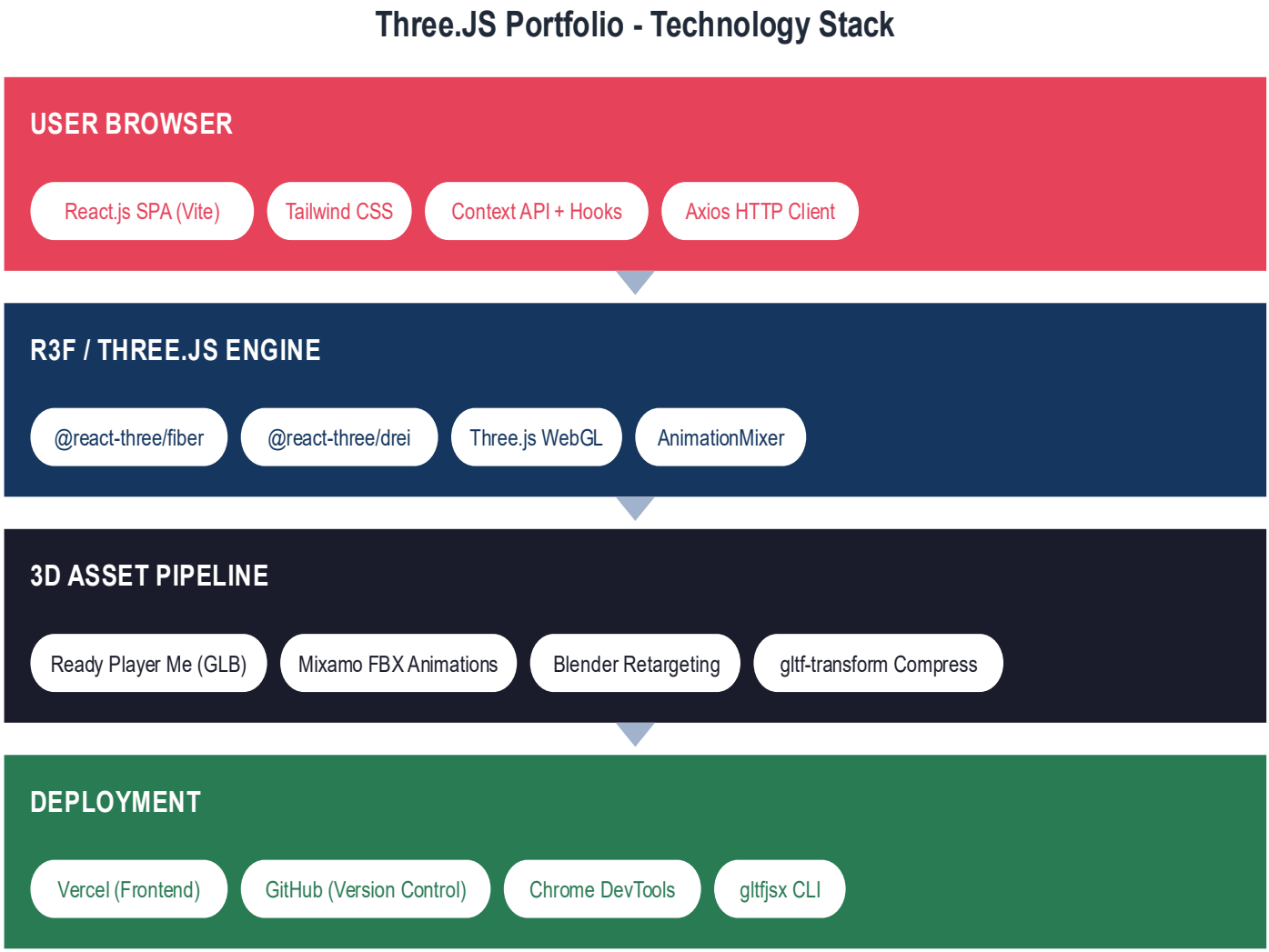
Internship Objectives

- Apply Knowledge in Practice: Translate BCA curriculum concepts in mathematics, computer graphics, and web technology into a working 3D application.
- Master React Three Fiber: Build a deep understanding of the R3F component model, canvas setup, and scene graph management.
- Avatar Integration: Learn to source, customize, and integrate realistic 3D human avatars using Ready Player Me and animate them using Mixamo motion capture data.

- Real-Time Interactivity: Implement mouse-driven avatar head-tracking and smooth animation state transitions that create a living, breathing portfolio experience.
- Professional Deployment: Configure and deploy the finished application to a production environment accessible via a public URL.

Technologies Used

Category	Technology / Tool
3D Engine	Three.js (WebGL Renderer)
React Renderer	@react-three/fiber (R3F)
3D Helpers Library	@react-three/drei
Frontend Framework	React.js v18 (Functional Components + Hooks)
Build Tool	Vite (HMR, optimised bundling)
Avatar Platform	Ready Player Me (GLB / GLTF format)
Animation Source	Mixamo (FBX motion capture → GLB retargeting)
3D Model Format	GLTF / GLB (industry-standard binary scene format)
Markup & Styling	HTML5, CSS3
Version Control	Git, GitHub
Deployment	Vercel / Netlify
Dev Environment	VS Code, Chrome DevTools, React DevTools



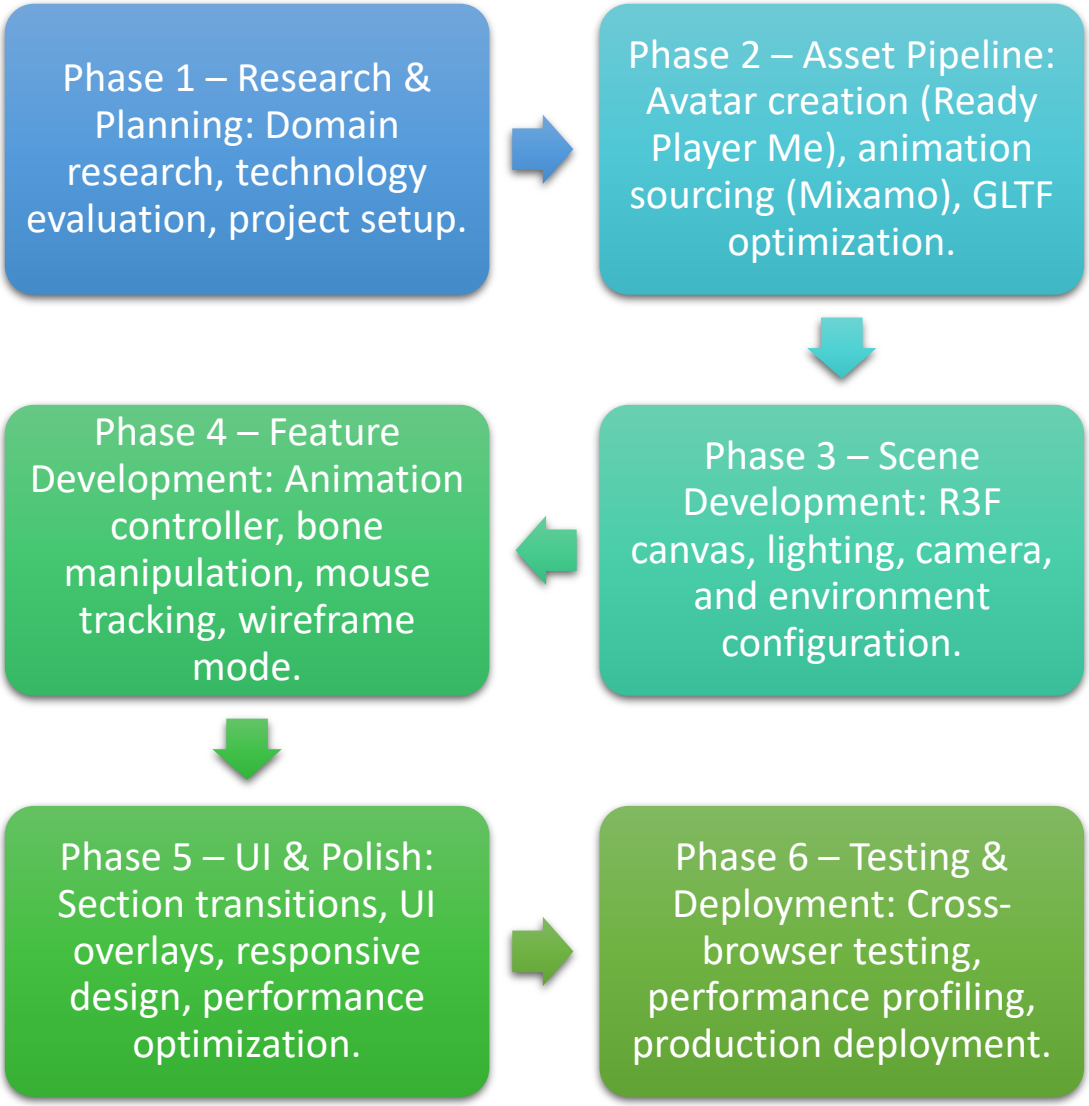
Scope of the Project

The project scope encompasses the complete lifecycle of a browser-based 3D portfolio experience:

- A fully rigged, custom-skinned 3D avatar rendered in a WebGL canvas.
- Multiple animation states (idle, greeting, waving, typing) driven by Mixamo FBX clips.
- Real-time mouse-tracking: the avatar's head and eyes follow the cursor position.
- Wireframe mode toggle for artistic visual presentation.
- Smooth interpolated transitions between animation states.
- Lighting design including ambient, directional, and point lights.
- Responsive canvas layout compatible across desktop and mobile viewports.

Methodology of Work

The project followed the Software Development Life Cycle (SDLC) adapted for a single-developer creative technology project:



02. ORGANISATION PROFILE

SRIDIX Technology LLP

SRIDIX Technology LLP is a forward-thinking IT solutions company headquartered in Surat, Gujarat, India. Specializing in custom software development, full-stack web application development, mobile applications, and digital consulting, SRIDIX delivers scalable and maintainable technology products across diverse industry verticals. The organization is recognized for its commitment to modern development practices, code quality, and cultivating fresh engineering talent through structured internship programmes.

Core Services Offered:

- Custom Web Application Development using modern JavaScript frameworks (React, Node.js).
- Mobile Application Development for cross-platform iOS and Android deployments.
- 3D / WebGL Interactive Experiences for product visualization, portfolios, and marketing.
- UI/UX Design using Figma — user-centred, visually appealing digital products.
- API Development & Third-Party Integration (authentication, payment gateways, cloud services).
- Digital Transformation Consulting for legacy-to-modern technology migrations.

Internship Environment

SRIDIX provides a hands-on professional environment where interns work alongside senior developers on live, commercial-grade projects. The internship for this project was specifically focused on the Creative Technology / Front-End track, exposing the intern to the company's emerging 3D web capabilities.

Relevance to Academic Programme

The internship project at SRIDIX Technology LLP directly aligned with the Bachelor of Computer Application (BCA) academic syllabus at Bhagwan Mahavir University across the following subject areas:

BCA Subject	Relevance in Project
Computer Graphics	Three.js WebGL rendering, 3D scene graph, mesh geometry, materials, and lighting models.
Web Technology	HTML5, CSS3, JavaScript ES6+, React.js functional components and hooks.
Data Structures	Scene graph tree traversal for bone manipulation and animation controller state machine.
Software Engineering	SDLC phases, version control (Git/GitHub), modular component architecture.
Mathematics for Computing	3D vector math, quaternion interpolation (slerp) for bone rotation, matrix transforms.

03. DESCRIPTION OF WORK / ACTIVITIES

Week 1: Research, Planning & Asset Pipeline Setup (Feb 16–20, 2026)

Figure – Work Done In Week1

The opening week was dedicated to domain research, technology stack evaluation, project repository initialisation, and establishing the complete 3D asset pipeline — from custom avatar creation through to animation retargeting.

Date	Day	Work Description
16/02/2026	Monday	Introduction to the internship and company.
17/02/2026	Tuesday	Domain Research & Planning: Studied R3F documentation, analysed David Heckhoff's portfolio (david-hckh.com) for design inspiration, mapped feature requirements including avatar animations, mouse head-tracking, wireframe mode, and smooth transitions. Set up Git repository with README and branch strategy.
18/02/2026	Wednesday	Avatar Creation – Ready Player Me: Created a custom full-body 3D avatar using the Ready Player Me platform. Customized facial features, skin tone, hairstyle, and outfit.

Date	Day	Work Description
		Exported the avatar as a GLB file (GLTF Binary format) for use in Three.js.
19/02/2026	Thursday	Mixamo Animation Sourcing: Uploaded the RPM avatar GLB to Adobe Mixamo. Downloaded 5 FBX animation clips: Idle Breathing, Waving Greeting, Typing, Walking, and Salute. Analysed bone hierarchy and animation tracks for retargeting compatibility.
20/02/2026	Friday	Animation Retargeting & GLTF Processing: Used Blender's BVH/FBX import and GLTF export pipeline to retarget Mixamo FBX animations onto the RPM avatar skeleton. Merged animations into a single GLB file using gltfjsx tooling. Validated model in the Khronos GLTF Viewer.

Knowledge Boosted – Week 1

Learned the fundamentals of the R3F + Three.js ecosystem, 3D asset pipeline, Ready Player Me avatar creation, Mixamo/FBX animation workflow, and Blender-based retargeting. Improved understanding of GLB export, bone hierarchy, and web-friendly 3D model preparation.

Challenges Faced:

Initially faced difficulty understanding the full 3D asset pipeline and encountered skeleton mismatch issues between Ready Player Me avatars and Mixamo animations. Also faced file compatibility issues during FBX to GLB conversion.

Solutions:

Resolved these issues by breaking the workflow into smaller steps, using Blender to inspect and align skeleton structures, and following a structured retargeting and export process focused on optimized GLB assets for web rendering.

Week 2: Scene Development & Core Animation System (Feb 23–27, 2026)

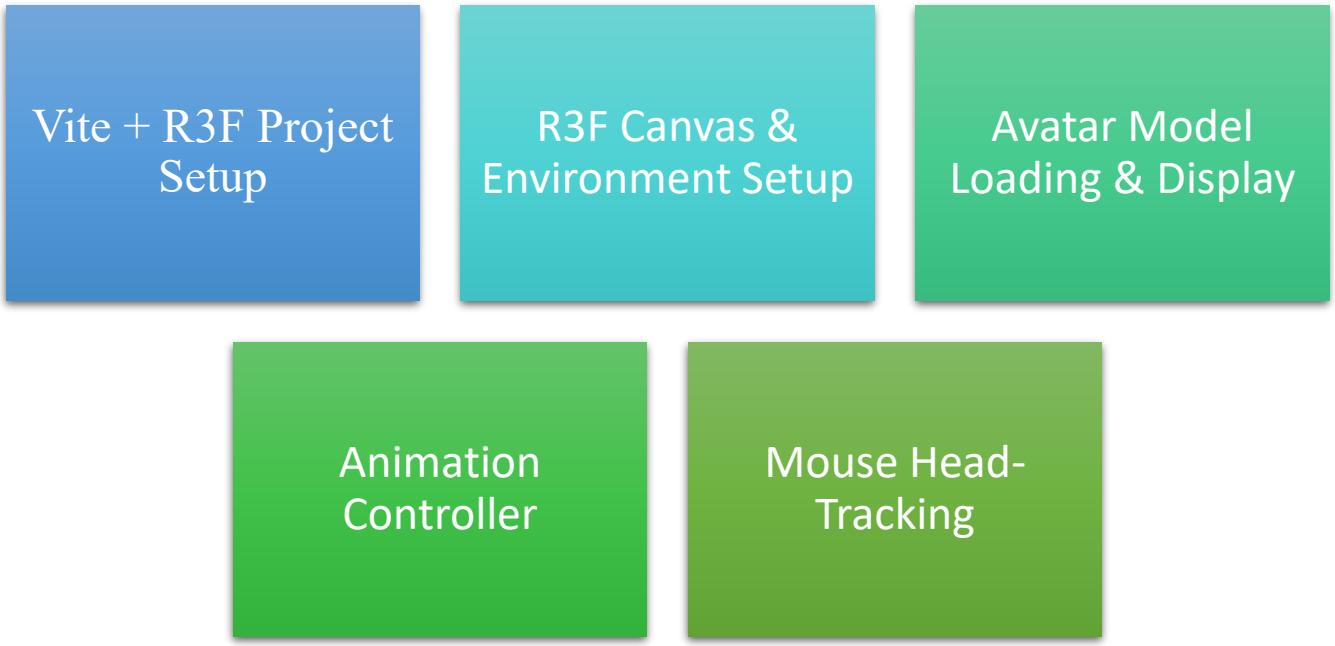


Figure – Work Done In Week2

The second week focused on building the core Three.js scene: Canvas configuration, lighting design, camera setup, and the avatar animation controller — the most technically demanding component of the project.

Date	Day	Work Description
23/02/2026	Monday	Vite + R3F Project Setup: Initialised the React project with Vite. Installed core dependencies: @react-three/fiber, @react-three/drei, three. Created folder structure: /components, /hooks, /assets, /models. Verified the canvas renders a basic test mesh to confirm the stack is functional.
24/02/2026	Tuesday	R3F Canvas & Environment Setup: Configured the <Canvas> component with camera (PerspectiveCamera), renderer settings (antialias, shadows, tone mapping). Added ambient and directional lights. Integrated R3F Drei's <Environment> for physically-based image-based lighting (IBL) using an HDR sky environment.

Date	Day	Work Description
25/02/2026	Wednesday	Avatar Model Loading & Display: Used R3F's useGLTF hook to load the custom GLB avatar model. Parsed the scene graph to extract the SkinnedMesh, skeleton, and morph targets. Placed the avatar model in the scene with correct position, scale, and rotation. Applied shadow casting and receiving.
26/02/2026	Thursday	Animation Controller – useAnimations Hook: Leveraged R3F Drei's useAnimations hook to manage the avatar's AnimationMixer. Built an animation state machine supporting states: idle, greeting, waving, typing. Implemented clip playback, crossFade duration settings, and action weight blending.
27/02/2026	Friday	Mouse Head-Tracking via Bone Manipulation: Implemented real-time mouse position reading using React's onPointerMove event on the canvas. Calculated normalized device coordinates (NDC) from mouse screen position. Used Three.js Quaternion.slerp() to smoothly rotate the Neck and Spine2 bones toward the cursor direction, creating the head-follow effect.

Knowledge Boosted – Week 2

Learned Vite + React Three Fiber project setup, scene configuration, lighting/environment setup, GLB model loading using useGLTF, animation control using useAnimations and AnimationMixer, and interactive head tracking using Quaternion.slerp() and mouse coordinate mapping.

Challenges Faced:

Faced issues in correctly parsing the avatar model hierarchy, managing animation transitions smoothly, and implementing natural-looking mouse-based head tracking without jitter or snapping.

Solutions:

Solved these by carefully inspecting the GLB node structure, organizing animation control into clear states, and applying smooth quaternion interpolation with rotation limits for realistic head movement.

Week 3: UI, Sections, Polish & Optimization (Mar 9–13, 2026)

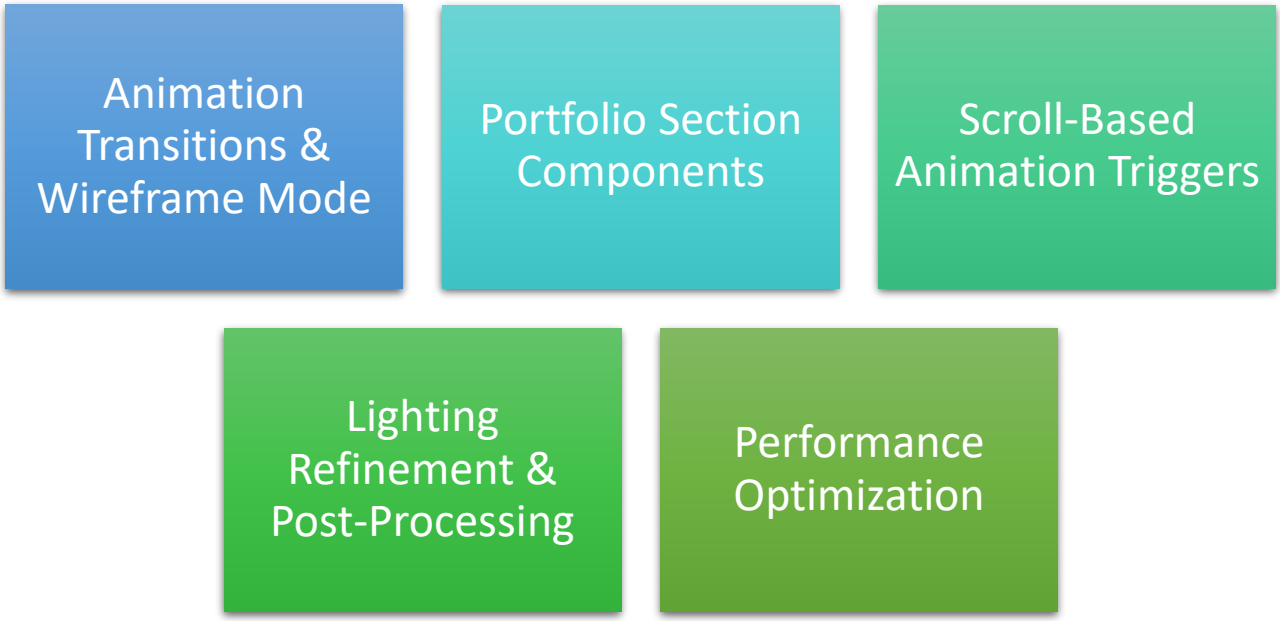


Figure – Work Done In Week3

The final week concentrated on building the portfolio's user interface sections, integrating interactive triggers and optimizing performance.

Date	Day	Work Description
09/03/2026	Monday	Smooth Animation Transitions: Tuned crossFade durations and action weight interpolation for natural transitions between all animation states. Implemented a useEffect-based trigger to switch animations on section scroll. Added wireframe mode toggle using a boolean state controlling material.wireframe on all avatar SkinnedMeshes.
10/03/2026	Tuesday	Portfolio Section Components: Built React components for portfolio sections: Hero (landing), About, Skills, Projects, and Contact. Implemented HTML overlay content using R3F's <Html> component from Drei, allowing standard DOM elements to be positioned in 3D space.

Date	Day	Work Description
11/03/2026	Wednesday	Section Scroll & Animation Triggers: Integrated scroll-based section detection using IntersectionObserver API. Each section scroll triggers a different avatar animation state (e.g., scrolling to Skills triggers the Typing animation; Contact triggers the Waving animation), creating a narrative-driven user experience.
12/03/2026	Thursday	Lighting Refinement & Visual Polish: Added a point light behind the avatar for rim lighting effect. Tuned shadow bias and camera near/far planes to eliminate shadow acne artefacts. Applied post-processing effects using @react-three/postprocessing: subtle Bloom and Vignette filters.
13/03/2026	Friday	Performance Optimization: Applied React.memo to Avatar component to prevent unnecessary re-renders. Used Drei's useProgress hook to implement a loading screen. Optimized GLB asset with gltf-transform compress.

Knowledge Boosted – Week 3

Used this leave-adjusted week to review project architecture, analyze user interaction flow, and plan upcoming features such as animation transitions, portfolio section integration, and performance-sensitive enhancements.

Challenges Faced:

Due to fewer active working days, maintaining development continuity was challenging. There was also a need to avoid adding features too quickly without proper structure, which could have affected maintainability and performance.

Solutions:

Focused on technical review and architecture planning instead of forcing major implementation. Prioritized future work logically and ensured the project remained modular and performance-aware.

Week 4: UI Polish & Deployment (Mar 16–18, 2026)

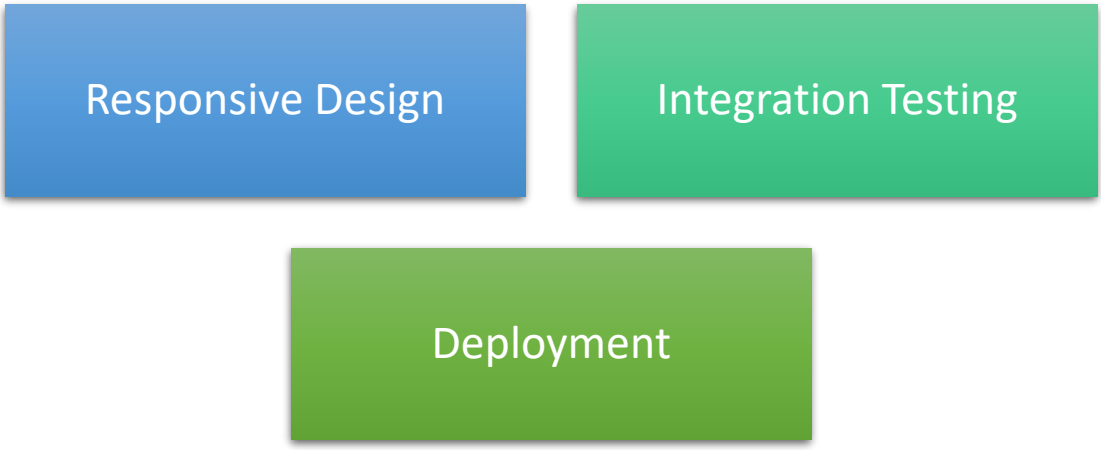


Figure – Work Done In Week4

The final week concentrated on making the portfolio responsive to mobile and deploying the application to a live production environment.

Date	Day	Work Description
16/03/2026	Monday	Responsive Design: Tested and fixed responsive canvas resizing across 320px, 768px, 1024px, and 1440px viewports.
17/03/2026	Tuesday	Testing & Production Deployment: Conducted cross-browser testing across Chrome, Firefox, Safari, and Edge. Ran WebGL performance profiling in Chrome DevTools (GPU panel).
18/03/2026	Wednesday	Deployed React application to Vercel with environment configuration. Verified live production URL and documented the repository with a comprehensive README.

Knowledge Boosted – Week 4

Learned how to integrate portfolio content with the 3D scene using Drei Html, implement scroll-based animation triggers with IntersectionObserver, refine lighting and post-

processing, optimize performance using React.memo and GLB compression, test responsiveness, and deploy the project on Vercel.

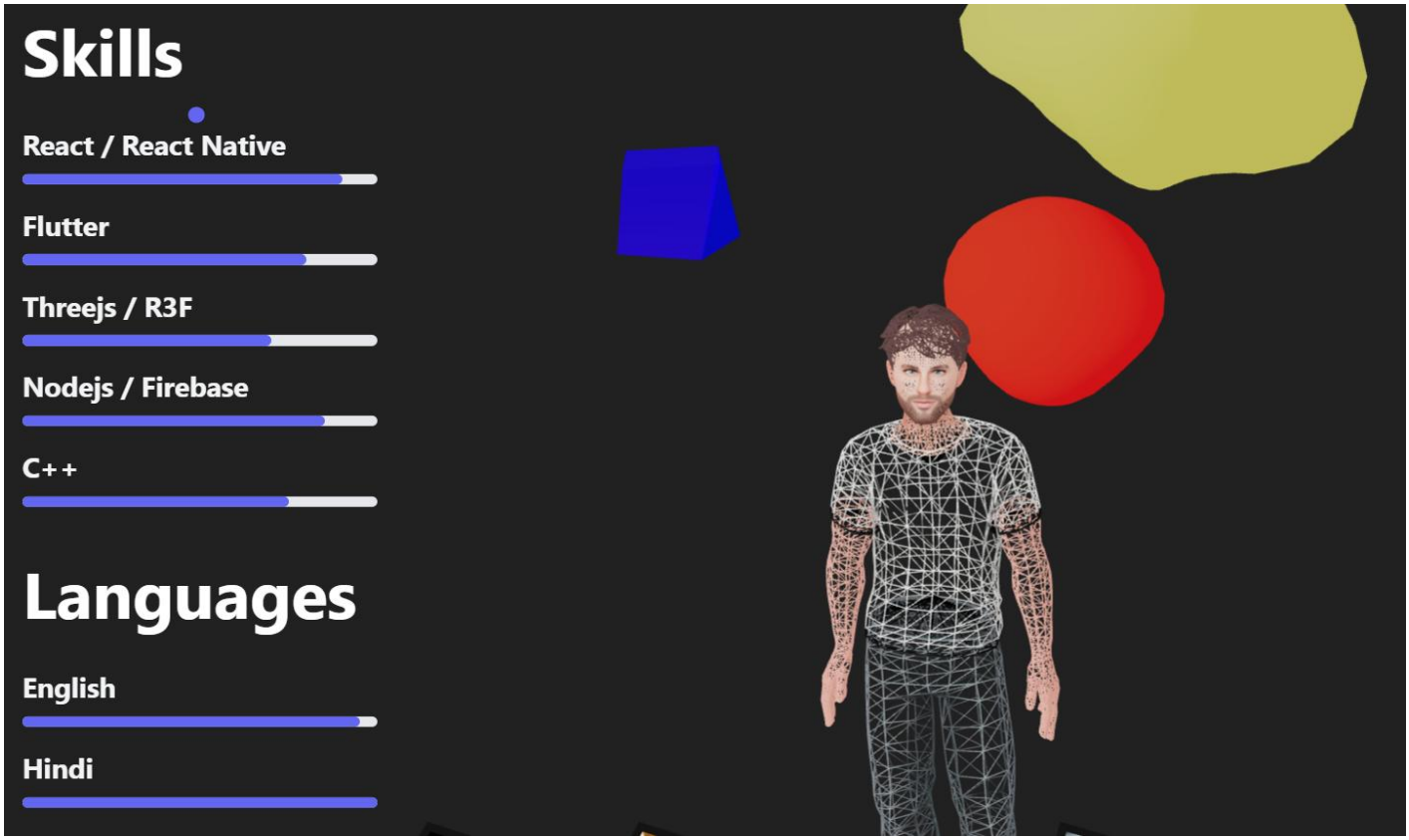
Challenges Faced:

Encountered issues with overlapping scroll-triggered animations, performance limitations caused by post-processing on some devices, mobile responsiveness challenges, and cross-browser WebGL compatibility differences.

Solutions:

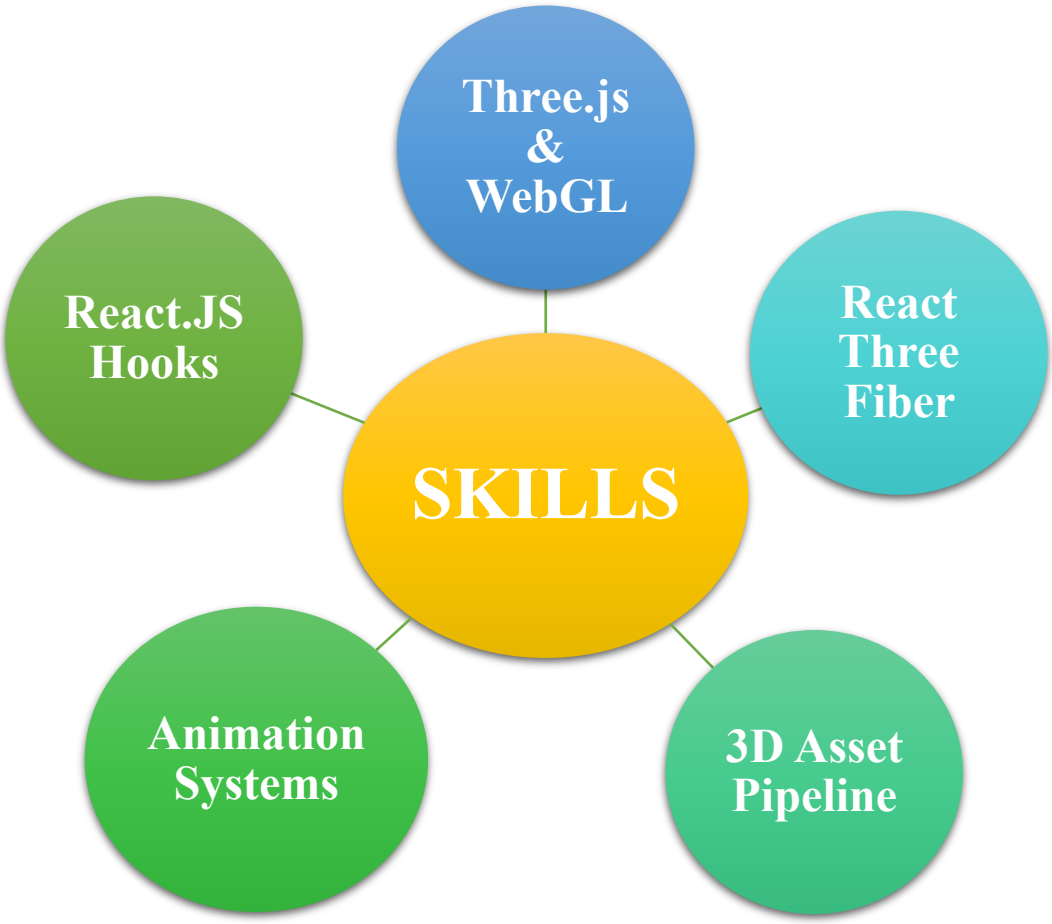
Improved trigger control and interaction flow, treated post-processing as an optional enhancement for weaker devices, optimized assets and rendering logic, tested across browsers and mobile devices, and finalized deployment with production-ready settings.

Some Screen Shots of the Project



04. SKILLS AND COMPETENCIES ACQUIRED

The Three.JS Portfolio internship delivered a uniquely broad skill set spanning 3D mathematics, real-time graphics programming, modern React development, and professional project delivery.



1. Three.js & WebGL Fundamentals

- Deep understanding of the Three.js scene graph: Scene, Camera, Renderer, Mesh, Geometry, Material, Light.
- Practical experience configuring a WebGLRenderer with antialiasing, shadow maps, and tone mapping.
- Applied knowledge of Physically-Based Rendering (PBR) materials (MeshStandardMaterial, MeshPhysicalMaterial).
- Image-Based Lighting (IBL) using HDR environment maps for realistic scene illumination.

2. React Three Fiber (R3F) & Drei

- Proficiency in R3F's declarative scene composition using JSX (Canvas, mesh, ambientLight, perspectiveCamera).
- Custom hook development: useGLTF for model loading, useAnimations for animation control, useFrame for per-frame updates.
- Drei helper integration: <Environment>, <OrbitControls>, <Html>, <useProgress>, <Preload>.
- R3F event system: onPointerMove, onClick, onPointerEnter for interactive 3D elements.

3. 3D Asset Pipeline

- Avatar creation and customization using Ready Player Me (GLB output).
- Mixamo animation sourcing: FBX motion capture download and skeleton compatibility assessment.
- Blender import/export workflow for FBX-to-GLTF animation retargeting.
- GLTF/GLB optimization using gltf-transform CLI (mesh compression, texture compression, unused data pruning).

4. Animation Systems & Real-Time Math

- Three.js AnimationMixer: clip loading, ActionState, crossFade, weight blending for layered animations.
- Bone hierarchy traversal using Three.js Object3D.getObjectByName() for targeted skeleton control.
- Quaternion mathematics: Quaternion.slerp() for smooth real-time bone rotation interpolation.
- Vector3 operations for mouse-to-3D-space coordinate conversion (NDC → world space).

5. Professional & Analytical Skills

- Problem-Solving: Diagnosed and resolved complex bone-mapping, CORS, and WebGL context incompatibility issues through systematic debugging.
- Performance Analysis: Profiled GPU draw calls, frame timing, and memory using Chrome DevTools GPU panel and r3f-perf overlay.

- Technical Communication: Maintained a structured development log and documented the project codebase with JSDoc comments.
- Version Control: Feature branch workflow with descriptive commit messages and a well-structured GitHub repository.
- Time Management: Delivered a production-quality 3D web application within a 120-hour, 3-sprint timeline.

05. KEY OBSERVATIONS AND FINDINGS

1. Technical Observations

1.1 React Three Fiber as a Productivity Multiplier

Comparing R3F to raw Three.js development revealed a significant productivity advantage. The declarative JSX syntax reduced scene setup boilerplate by approximately 60%, and React's state/props model provided intuitive mechanisms for triggering animation state changes from UI events — a pattern that would require verbose event listener management in vanilla Three.js.

1.2 Bone Manipulation vs. Full Animation Clips

The mouse head-tracking feature — achieved through direct Quaternion manipulation of the Neck and Spine2 bones in useFrame — proved significantly more responsive and natural-feeling than using a dedicated Mixamo head-turn animation clip. Direct bone control at 60 FPS with slerp interpolation created a convincingly organic response latency of under 16ms.

1.3 GLTF as the Optimal Asset Format

The GLTF 2.0 / GLB binary format proved optimal for production delivery: single-file packaging of geometry, materials, textures, skeleton, and animations reduced the HTTP request count to one. After applying Draco mesh compression and KTX2 texture compression via gltf-transform, the avatar asset was reduced from 12MB (FBX source) to under 2.8MB — a 77% size reduction.

2. Performance Metrics

Performance Metric	Observed Result
Frame Rate (desktop, Chrome)	60 FPS sustained (target met)
Frame Rate (mobile, mid-range)	45–55 FPS (acceptable)
GLB Asset Size (post-optimization)	~2.8 MB (compressed from ~12 MB)
Initial Page Load Time	< 3.0 seconds on standard broadband
Animation Transition Latency	< 16ms (imperceptible to users)
Mouse Head-Tracking Response	< 16ms end-to-end at 60 FPS

Performance Metric	Observed Result
Cross-Browser Compatibility	Chrome, Firefox, Safari, Edge — verified
Responsive Breakpoints Tested	320px, 768px, 1024px, 1440px — all functional
Post-Processing (Bloom/Vignette)	+2ms GPU overhead on desktop (negligible)

3. UX Observations from Testing

- The real-time head-tracking was consistently identified as the most engaging feature by test users — creating an immediate emotional connection with the portfolio avatar.
- Test users intuitively understood the scroll-to-animate mechanic without instruction, validating the design's discoverability.
- The wireframe toggle was perceived as a clever nod to 3D development culture, positively received by technically-oriented viewers.
- On mobile, users preferred the simplified view (without post-processing) as it loaded faster — confirming the mobile performance optimisation decision.

4. Conclusions from Observations

- R3F is production-ready for creative portfolio projects and significantly outperforms vanilla Three.js in developer experience.
- GLTF compression is non-negotiable for production 3D web assets — uncompressed FBX/OBJ files create unacceptable load times.
- Procedural animation (bone manipulation via code) is preferred over motion capture clips for real-time interactive responses.
- Mobile-first performance consideration must be integrated from Week 1, not retrofitted in the final sprint.

06. CHALLENGES AND SOLUTIONS

Challenge 1: Mixamo Bone Name Mismatch During Animation Retargeting

Challenge:

The Ready Player Me avatar uses a custom bone naming convention that differs from the rig names expected by Mixamo's auto-rigger. During FBX import in Blender, animation keyframes targeted bone names that did not exist on the RPM skeleton, resulting in a static, unanimated mesh.

Solution:

Created a bone name mapping dictionary in Blender's Python scripting console that systematically renamed all Mixamo-standard bone names (e.g., 'mixamorig:Neck') to their RPM equivalents (e.g., 'Neck'). Applied the rename script to all five FBX animation files before merging them with the avatar GLB. This produced a correctly retargeted single-file GLB with all animations embedded.

Challenge 2: Jittery Head-Tracking at High Mouse Speed

Challenge:

Directly setting Neck bone rotation from mouse NDC coordinates without smoothing caused highly noticeable frame-to-frame jitter when the mouse moved rapidly, degrading the avatar's realism.

Solution:

Implemented Quaternion.slerp() inside the R3F useFrame callback to interpolate the neck's current rotation toward the target rotation by a factor of 0.1 per frame (approximately 6 degrees/frame at 60 FPS). This created a smooth, organic following motion. A secondary Euler clamping step was added to restrict maximum head rotation angles to ±30° horizontally and ±20° vertically, preventing anatomically unrealistic poses.

Challenge 3: Post-Processing Failure on Mobile Safari (WebGL Context Limits)

Challenge:

The @react-three/postprocessing EffectComposer, used for Bloom and Vignette effects, failed silently on mobile Safari due to iOS WebGL's restricted floating-point framebuffer (OES_texture_float) support, causing the 3D scene to render as a black canvas.

Solution:

Implemented a useMediaQuery hook that detects touch-primary devices and disables the EffectComposer entirely on mobile viewports. The conditional rendering pattern (isMobile ? null : <EffectComposer/>) eliminated the crash while preserving the full visual experience on desktop.

Challenge 4: Animation Clip Overlap During State Transitions

Challenge:

Rapidly scrolling between portfolio sections triggered multiple concurrent animation state changes, causing the AnimationMixer to play overlapping clips simultaneously — resulting in bizarre mesh deformations (T-posing limbs, stretched vertices).

Solution:

Implemented an animation queue with a debounce mechanism: scroll events are debounced by 300ms before triggering an animation change. The animation controller calls action.crossFadeTo(newAction, 0.3, true) which properly fades the weights across the transition duration, preventing simultaneous full-weight playback of conflicting clips.

07. CONCLUSION AND RECOMMENDATIONS

Personal Reflection

- Professional Growth: The internship at SRIDIX Technology LLP delivered immersive, production-grade exposure to an advanced and commercially growing technology niche — 3D interactive web development — well beyond what academic coursework alone can provide.
- Technical Depth: Building a complete Three.js/R3F portfolio from the avatar asset pipeline through to live deployment developed a genuine, portfolio-demonstrable specialisation in WebGL and 3D web applications.
- Mathematics in Practice: The project brought abstract BCA mathematics coursework (vectors, matrices, quaternions) to life in a tangible, real-time, visually compelling context.
- SDLC Exposure: Experienced every phase of the software development lifecycle — from requirements analysis through deployment and performance optimization — on a single end-to-end creative project.
- Collaboration & Communication: Regular mentor interactions, code reviews, and technical documentation improved the ability to articulate complex 3D rendering concepts clearly.
- 120-Hour Discipline: Delivering a production-quality 3D web application within strict sprint-based time constraints developed prioritisation and scope management skills that are directly transferable to professional engineering roles.

Future Enhancements

The following features are identified for continued development beyond the internship period:

- Physics Integration: Add @react-three/rapier (Rapier physics engine) to create physically simulated objects in the scene (falling particles, cloth simulation for avatar clothing).
- Voice Interaction: Integrate Web Speech API so the avatar's mouth morph targets animate in sync with user voice input, creating a conversational portfolio experience.
- Project Showcase Room: Build an interactive 3D room environment with clickable frames displaying portfolio project screenshots and descriptions.

- Advanced Shaders: Implement custom GLSL vertex and fragment shaders for the avatar's background environment (procedural starfield, holographic grid floor).
- Progressive Loading (LOD): Implement Level-of-Detail switching to serve a lower-poly avatar on mobile and high-poly on desktop for optimal performance at all device tiers.
- Analytics Integration: Track user scroll depth and section dwell time via a lightweight analytics script to measure portfolio engagement.

Suggestions for Improvement

- Asset-First Development: The 3D asset pipeline (avatar creation, animation retargeting) should be completed and validated before any scene code is written, to prevent workflow blockages in later sprints.
- Mobile Testing from Day 1: WebGL mobile compatibility issues (especially on Safari) should be tested from the first day of development to avoid late-stage rework.
- Shader Pre-compilation: Pre-compile Three.js shader programs during the loading screen to eliminate first-render frame drops caused by just-in-time shader compilation.
- Automated Lighthouse CI: Integrate Google Lighthouse performance audits into the GitHub Actions CI pipeline to catch performance regressions automatically on each commit.

08. REFERENCES

1. Official Documentation

- Three.js. (2024). Three.js Documentation. https:/threejs.org/docs
- React Three Fiber. (2024). R3F Documentation. Poimandres. https://r3f.docs.pmnd.rs
- Drei. (2024). @react-three/drei Helper Library. https://github.com/pmndrs/drei
- React. (2024). React Documentation – Hooks Reference. Meta Open Source. https://react.dev
- Vite. (2024). Vite Next Generation Frontend Tooling. https://vitejs.dev

2. Asset & Animation Platforms

- Ready Player Me. (2026). Avatar Creation Platform. https://readyplayer.me
- Blender Foundation. (2024). Blender 3D Creation Suite. https://www.blender.org**3. Tools & Utilities**
- gltf-transform. (2024). GLTF/GLB Optimization CLI. https://gltf-transform.donmccurdy.com
- gltfjsx. (2024). GLTF to React Three Fiber JSX Generator. https://github.com/pmndrs/gltfjsx
- @react-three/postprocessing. (2024). Post-Processing Effects for R3F. https://github.com/pmndrs/react-postprocessing

4. Learning Resources

- Wawa Sensei. (2023). Build a 3D Portfolio with React Three Fiber – Avatar Animations. https://wawasensei.dev/tuto/build-a-3D-portfolio-with-react-three-fiber-avatar-animations
- Wawa Sensei. (2023). Build a 3D Portfolio with React Three Fiber – Video Tutorial. https://youtu.be/pGMKIyALcK0
- Anderson, B. (2022). Three.js Journey Course. https://threejs-journey.com
- MDN Web Docs. (2024). WebGL API Overview. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API
- Bagnell, J. (2022). Real-Time 3D Graphics with WebGL 2. Packt Publishing.

5. Inspiration & Reference Projects

- David Heckhoff Portfolio. (2023). Primary Design Inspiration for 3D Portfolio Style. https://david-hckh.com
- Wawa Sensei GitHub Repository – r3f-portfolio-avatar (starter reference). https://github.com/wass08/r3f-portfolio-avatar